Natural Language Processing

Parameter Efficient Fine-Tuning (PEFT)

Worries in the LLM era, Eduard Hovy, CMU, Rocling 2024.

Look what an LLM can do!

Why can it do that?

I have no idea / that's future work / I've never thought about it

Look what an LLM cannot do!

Why not?

I have no idea / that's future work / I've never thought about it

I don't care about LLMs – here's what I did

Why are you doing that? Can't an LLM do it already?

I have no idea / that's future work / I've never thought about it

Three major directions for New NLP, Eduard Hovy, CMU, Rocling 2024.

NLP engineering: make LLMs usable

- Build smaller and cheaper LLMs
- Systematize prompt engineering
- Integrate language, images, and other media and functions

NLP applications: make LLMs useful

- Tune LLMs to domains and companies for enterprise processing
- Add functionality and agency in the world
- Tailor LLMs to people to be their personal daemons / amanuenses in everyday life

NLP Research: make LLMs understandable (or at least, be solid engineering)

- Fix the problems with LLMs
- Ger explanations how LLMs do what they do
- Formalize them well enough for autonomy, assurance, and ethics

https://techcrunch.com/2024/10/31/openai-ceo-sam-altman-says-lack-of-compute-is-delaying-the-companys-products/

AI

OpenAI CEO Sam Altman says lack of compute capacity is delaying the company's products

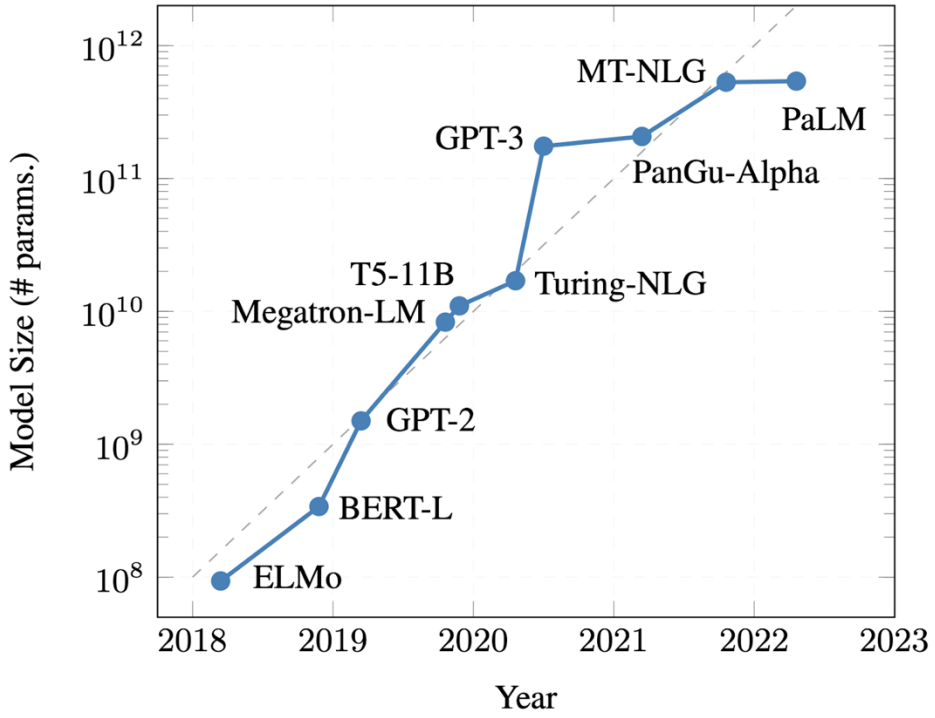
Outline

1. PEFT Introduction
2. PEFT Theory
   - Intrinsic Dimensionality
3. PEFT Current Development & Method
   - Adapters
   - Prompt Tuning / Prefix Tuning
   - Bitfit
   - LoRA 用小矩陣進行update
   - MAM Adapters
   - S4

PEFT Introduction

使用自己的資料集，有效率地進行模型微調

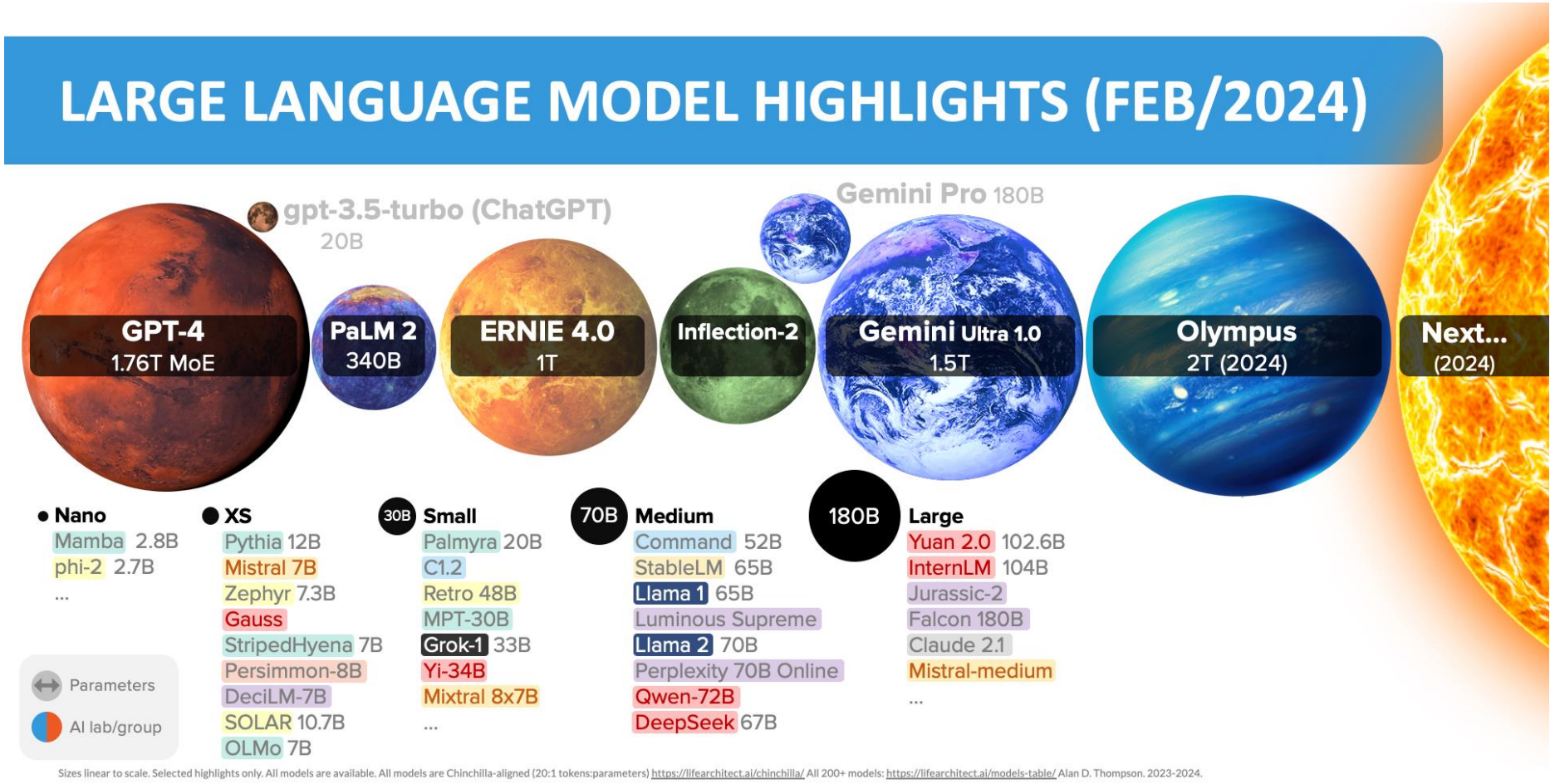
LLM Full Finetune Predicament



Model Name	Model Size	Developer
PaLM	540 Billion	Google
MT-NLG	530 Billion	Microsoft NVIDIA
PanGu-α	200 Billion	PengCheng
GPT-3	175 Billion	OpenAI
Turing-NLG	17.2 Billion	Microsoft

Treviso, Marcos, et al. "Efficient methods for natural language processing: A survey." *Transactions of the Association for Computational Linguistics* 11 (2023): 826-860.

LLM Full Finetune Predicament



Source: Inside language models (from GPT-4 to PaLM) – Dr Alan D. Thompson – Life Architect

Large Language Model (LLM) enabled NLP applications

Pre-train

xB ~ xxxB 參數

BERT
GPT
ChatGPT
GPT-4

Fine-tune

加入校正資料

doc summary → 文件摘要模型

doc Question/ answer → 問答系統

docs Retrieval / QA → 知識問答系統

GPU Memory Estimated – Full Finetune

大模型裡面的所有參數都需要調整

Llama 2-7B, 16-bit float, seq 4096		
CUDA		~1Gb
Model weights	size(float) * $N_{parameter}$	13.03Gb
Gradients	size(float) * $N_{trainable}$	13.03Gb
Hidden states	~size(float) L (20 seq + 3 seq^2)	3.16Gb (batch size = 1)
Optimizer states	2 * size(float) * $N_{trainable}$	26.06Gb

L : Number of layers in model (eq. 32 layers)
H : Number of attention heads (eq. 32 heads)

Estimate: 56.28Gb

Formula for Hidden States Estimate

During training:

$$(3\ h\ seq + L(4\ h\ seq + 3\ h\ seq^2 + 8\ h\ seq + 2\ h\ seq + 4\ h\ seq) + vocab\ seq)\ bs =$$

Emb,pos, Pre-logit h	K,V,Q,O	Score, Probs, Dropout	FFN hidden, activation	FFN out, dropout	residual, LN	logits

$$= 3\ h\ seq\ bs + 18L\ h\ seq\ bs + 3L\ heads\ seq2 + vocab\ seq\ \ bs$$

L : Number of layers in model (eq. 32 layers)
H : Number of attention heads (eq. 32 heads)
bs : batch size

https://github.com/hiyouga/LLaMA-Factory?tab=readme-ov-file#hardware-requirement

Hardware

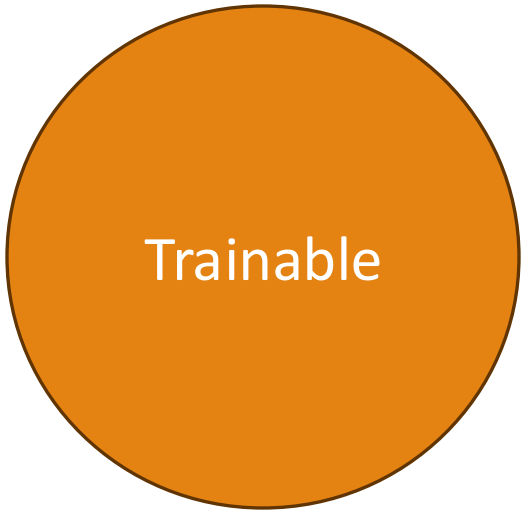
Hardware Requirement

* *estimated*

Inference need only 24GB, when batch=1, context len=4k

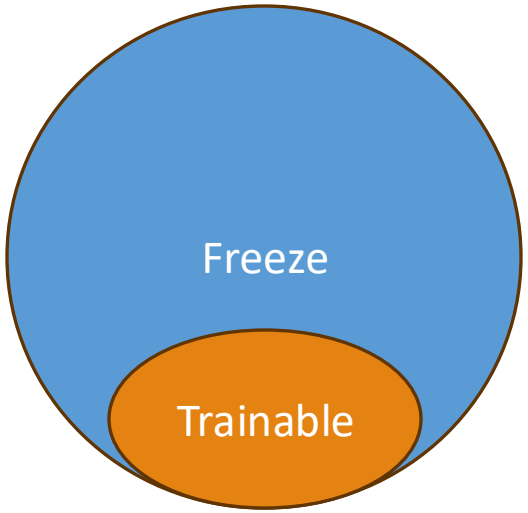
Method	Bits	7B	13B	30B	70B	110B	8x7B	8x22B
Full	AMP	120GB	240GB	600GB	1200GB	2000GB	900GB	2400GB
Full	16	60GB	120GB	300GB	600GB	900GB	400GB	1200GB
Freeze	16	20GB	40GB	80GB	200GB	360GB	160GB	400GB
LoRA/GaLore/BAdam	16	16GB	32GB	64GB	160GB	240GB	120GB	320GB
QLoRA	8	10GB	20GB	40GB	80GB	140GB	60GB	160GB
QLoRA	4	6GB	12GB	24GB	48GB	72GB	30GB	96GB
QLoRA	2	4GB	8GB	16GB	24GB	48GB	18GB	48GB

From Fine-tuning to Parameter-efficient Fine-tuning



Full Fine-tuning

Update **all model parameters**



Parameter-efficient Fine-tuning

Update a **small subset** of model parameters

只需要做小部分的參數訓練

GPU Memory Estimated – Train Less Parameters

Training only 0.2M parameters

Llama 2-7B, 16-bit float, seq 4096		
CUDA		~1Gb
Model weights	size(float) * $N^{parameter}$	13.03Gb (Same with full-FT)
Gradients	size(float) * $N^{trainable}$	**0.4Mb**
Hidden states	~size(float) L (20 H seq + 3 seq^2)	3.16Gb (batch size = 1) (Same with full-FT)
Optimizer states	2 * size(float) * $N^{trainable}$	**0.8Mb**

L : Number of layers in the model (eq. 32 layers)
H : Number of attention heads (eq. 32 heads)

Estimate: 17.19Gb (56.28 GB for full-FT)

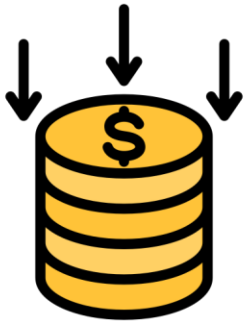
Haven't We Seen This Before ?

- Updating the last layer was common in computer vision
- In NLP, people experimented with static and non-static word embeddings
- ELMo did not fine-tune contextualized word embeddings

Matthew E. Peters, et al. "Deep Contextualized Word Representations." Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). 2018.

Benefits of PEFT

1. Decreased computational and storage costs
    - One significant benefit of parameter-efficient fine-tuning lies in its reduced computational and storage demands compared to the high costs associated with full fine-tuning.
2. Portability
    - It offers applicability across various domains and tasks. PEFT achieves this by **introducing a limited set of task-specific parameters while preserving the general-purpose parameters of the pre-trained model**, facilitating seamless transfer to new unlabeled datasets.

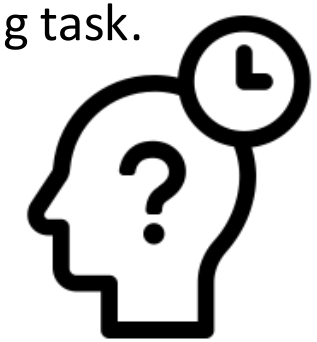


Benefits of PEFT

3. Overcoming catastrophic forgetting

災難式遺忘=> fine tune會進行全參數的update,導致以往的記憶點被覆蓋

- **When extensively pre-trained models undergo full fine-tuning on a novel task, it often leads to the model forgetting previously acquired knowledge** from its pre-training phase. The insights gained from the prior task are overridden by the updates tailored for the new task.
- Through PEFT, **only a small subset of parameters undergo modification during fine-tuning, leaving the majority of the model - which encapsulates general language knowledge from pre-training - unchanged**. This focused adjustment helps prevent the loss of knowledge from the original pre-training task.



Benefits of PEFT

4. Better performance in low-data regimes

資源少的時候，除了有可能會造成災難式遺忘，可能也會overfitting
=>不要使用full fine tune, 可以用PFT

- Large Language models of considerable size possess hundreds of millions or even tens of millions of trainable parameters, **thereby increasing the risk of overfitting when fine-tuned on tasks with limited labeled examples**.
- Nevertheless, **PEFT selectively trains only a small subset of these parameters, leveraging knowledge from the robust pre-trained model**. This approach enables the model to generalize more effectively as it still heavily relies on the extensive pre-trained representation.

Benefits of PEFT

5. Performance comparable to full fine-tuning
    - **Extensive studies have demonstrated that parameter-efficient techniques can match or surpass the performance achieved by fully fine-tuning pre-trained models**, despite adjusting only a minute fraction of parameters.
    - For instance, **research indicates that incorporating a small adapter module during fine-tuning yields performance results within 1% of fully adapting BERT across various natural language understanding benchmarks**. This illustrates that PEFT isn't just a workaround with compromised accuracy, but rather a method capable of achieving similarly robust outcomes while leveraging its numerous advantages over full fine-tuning.

Trainable Parameters Comparison

Method	RTE (Acc)	Trainable parameters (M)	Ratio
Full Fine-tune Acc	83.75%	184	100%
AdaLoRA	88.09%	1.27	0.69%
LoRA	86.60%	0.8	0.43%
Random Rank LoRA† **(rank : 1 - 16)**	85.56%	0.62	0.34%
Random Rank LoRA† **(rank : 8 - 24)**	85.16%	1.18	0.64%
Random Rank LoRA† **(rank : 16 - 32)**	85.13%	1.77	0.96%
Random Rank LoRA† **(rank : 32 - 64)**	84.91%	3.54	1.92%

† : Average of Samples (10 samples)
* Model: DeBERTa-v3-base
** Ratio means compare with Full Fine-tune

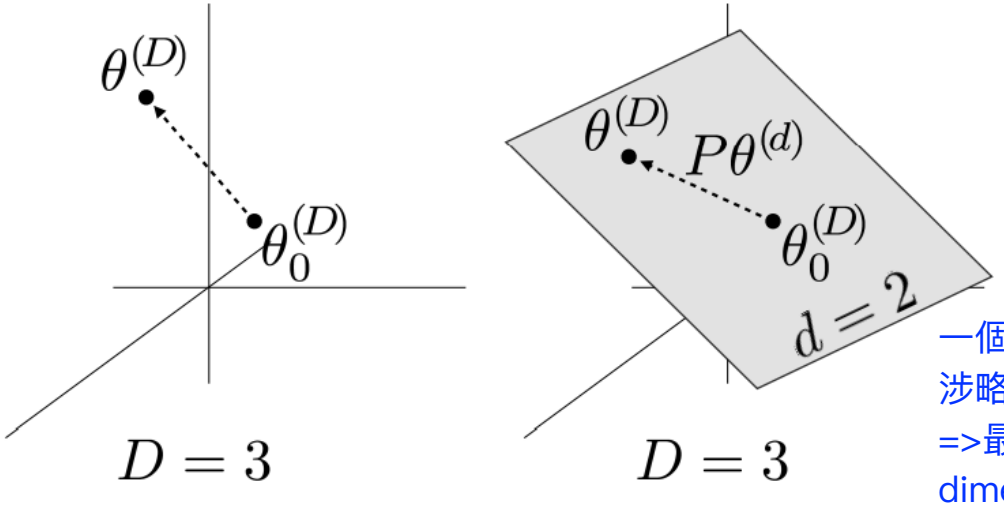
PEFT Theory

Intrinsic Dimensionality

一個LM有大量的模型參數，每train一次每個參數都要跑一次
=>PEFT:只要針對某個特定領域的內容，進行微調/fine-tuned model的策略

The definition of Intrinsic Dimensionality (d_{int})

- **Intrinsic dimensionality refers to the dimensionality of the solution set within the overall parameter space**
  - If a high-dimensional space (D = 1000), the effective dimensionality (**d_a** = 10) means we optimize by random subspace (d = 10) can achieve **"a %"** performance of original optimizing outcome.



一個模型執行時，實際需要涉略的維度？
=>最少要看多少dimension?就能完成任務

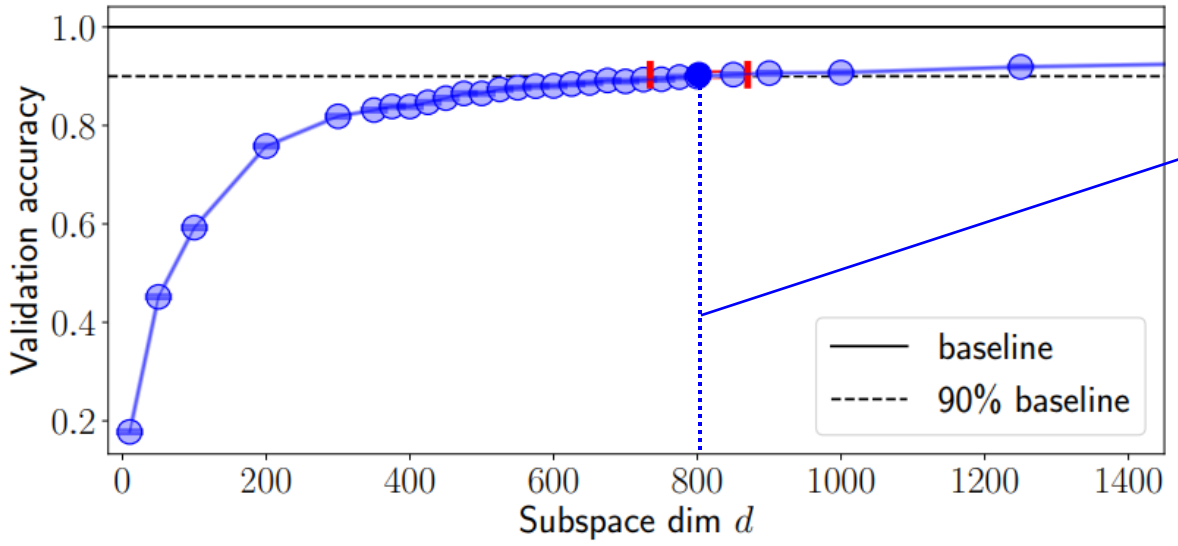
Li, Chunyuan, et al. "Measuring the Intrinsic Dimension of Objective Landscapes." *International Conference on Learning Representations*. 2018.

Intrinsic Dimensionality

Measuring the Intrinsic Dimension of Objective Landscapes

- Define $\mathbf{d_{100}}$ as the intrinsic dimension of the **"100%"** solution: solutions whose performance is statistically indistinguishable from baseline solutions

performance



dimension

In this example, the d_{90} is approximately equal to 800.

如何找出少數但重要的參數？
=> PEFT主要處理的問題

Li, Chunyuan, et al. "Measuring the Intrinsic Dimension of Objective Landscapes." *International Conference on Learning Representations*. 2018.

Intrinsic Dimensionality

Many problems have smaller intrinsic dimensions than one might suspect

Dataset	MNIST		CIFAR-10		Inverted Pendulum	Humanoid	Atrai Pong
Network Type	FC	LeNet	FC	LeNet	FC	FC	ConvNet
Parameter Dim. D	199,210	44,426	656,810	62,006	562	166,673	1,005,974
Intrinsic Dim. d_{90}	750	290	9,000	2,900	4	700	6,000
d_{90} / D	**0.38%**	**0.65%**	**1.37%**	**4.68%**	**0.7%**	**0.42%**	**0.60%**

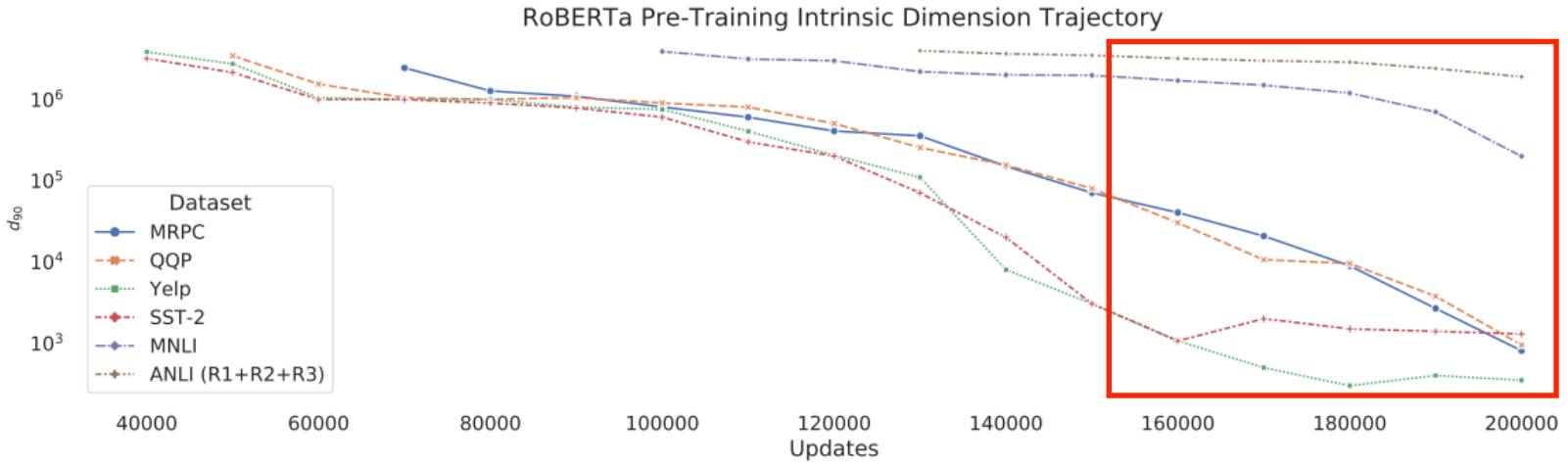
Li, Chunyuan, et al. "Measuring the Intrinsic Dimension of Objective Landscapes." *International Conference on Learning Representations*. 2018.

Intrinsic Dimensionality

Pre-training implicitly minimizes intrinsic dimension

- Compute d_{90} for six datasets: MRPC, QQP, Yelp Polarity, SST-2, MNLI, and ANLI
- See that **the intrinsic dimensionality of RoBERTa-base monotonically decreases as we continue pre-training**

在pre-train階段，提供越多的train dataset,
fine-tune階段就不需要update過多參數

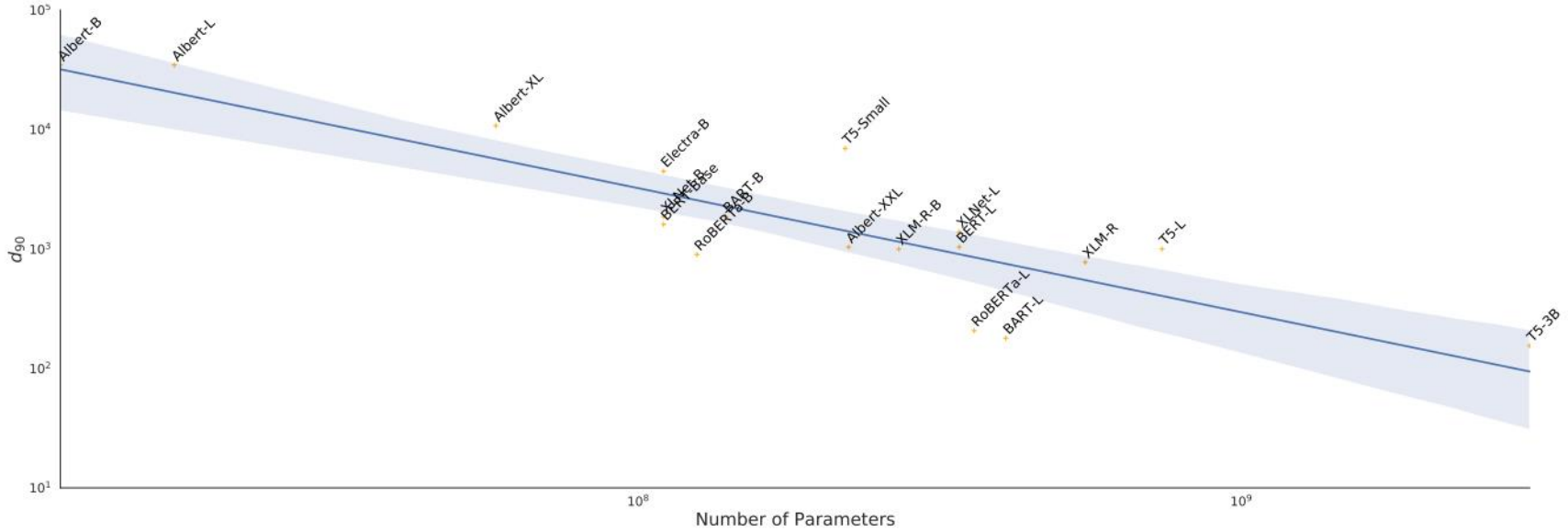


Aghajanyan, Armen, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning." *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2021.

Intrinsic Dimensionality

Larger models tend to have lower intrinsic dimension after a fixed number of pre-training updates

- Used the MRPC dataset and computed intrinsic dimension for every pre-trained model
- See a strong general trend that **as the number of parameters increases, the intrinsic dimension of fine-tuning on MRPC decreases**



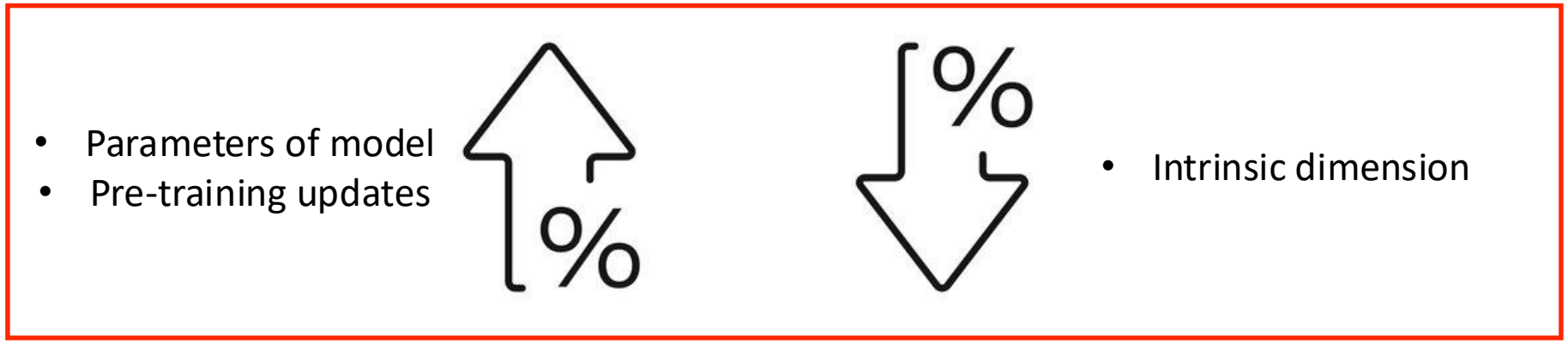
Aghajanyan, Armen, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning." *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2021.

Intrinsic Dimensionality

Observations

- Many problems have smaller intrinsic dimensions
- Intrinsic dimensionality decreases during pre-training
- Larger models have lower intrinsic dimensionality

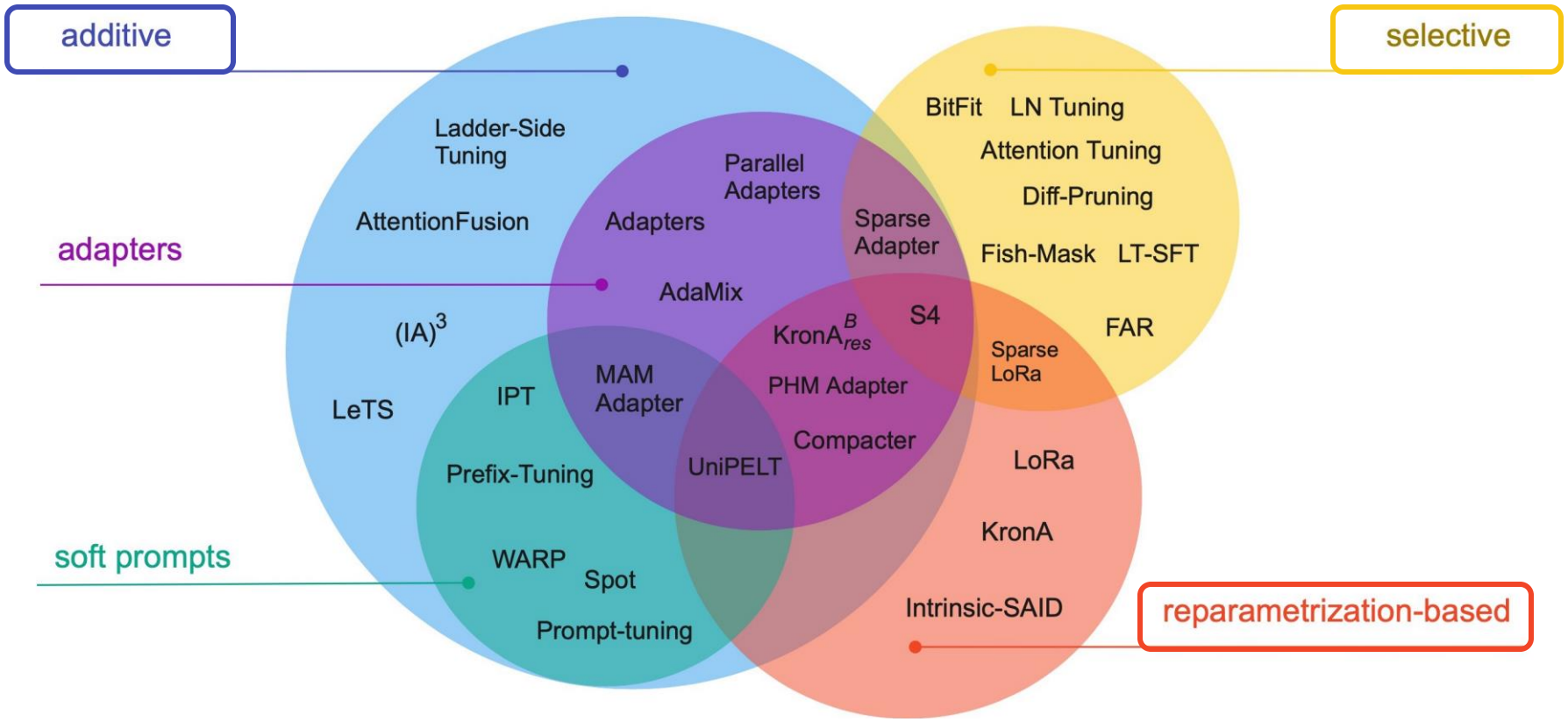
在越大的模型上做PEFT,成功機率越高



Aghajanyan, Armen, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning." *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2021.

PEFT
Current Development & Method

PEFT Current Development

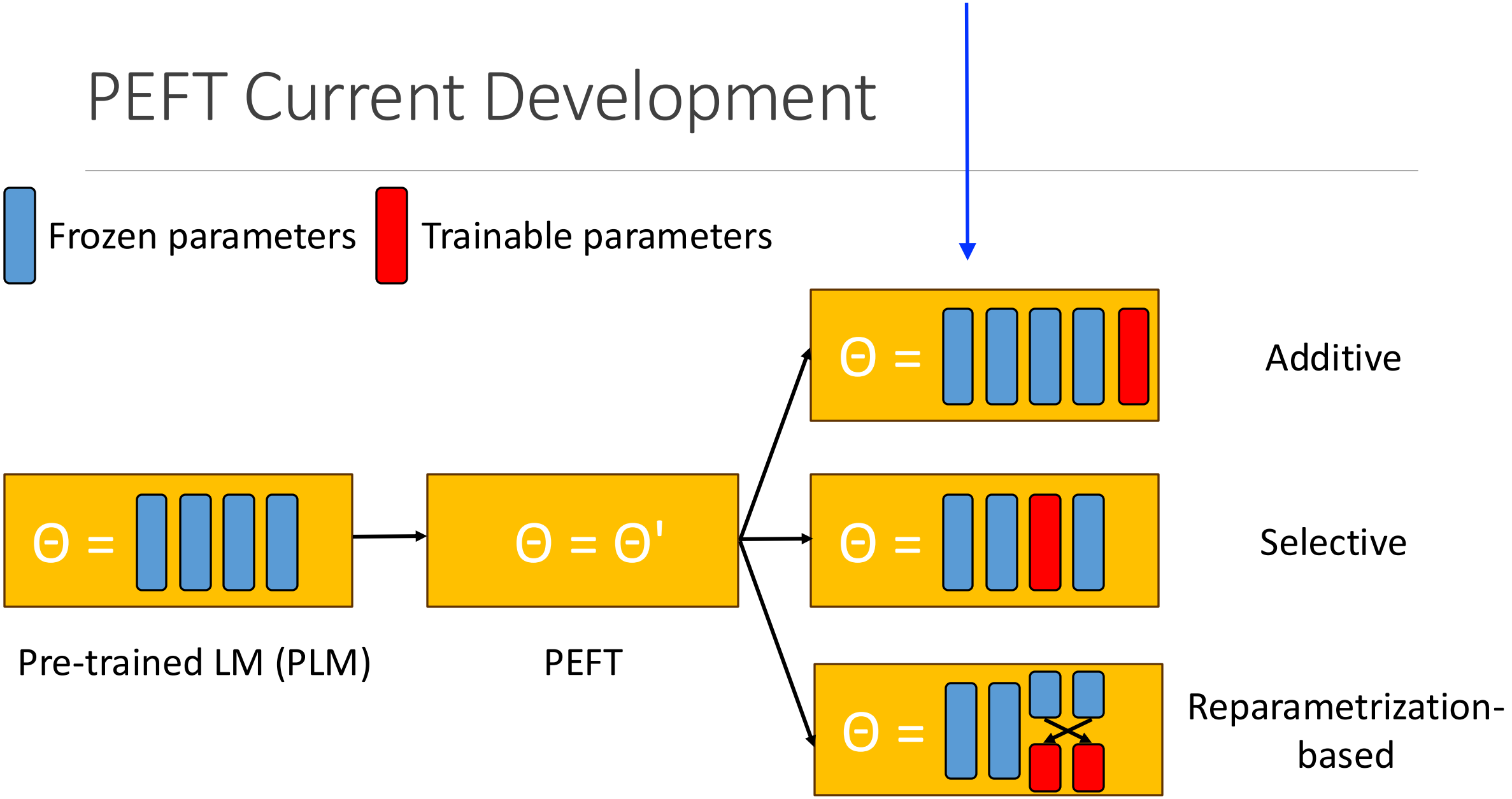


Lialin, Vladislav, Vijeta Deshpande, and Anna Rumshisky. "Scaling down to scale up: A guide to parameter-efficient fine-tuning." *arXiv preprint arXiv:2303.15647* (2023).

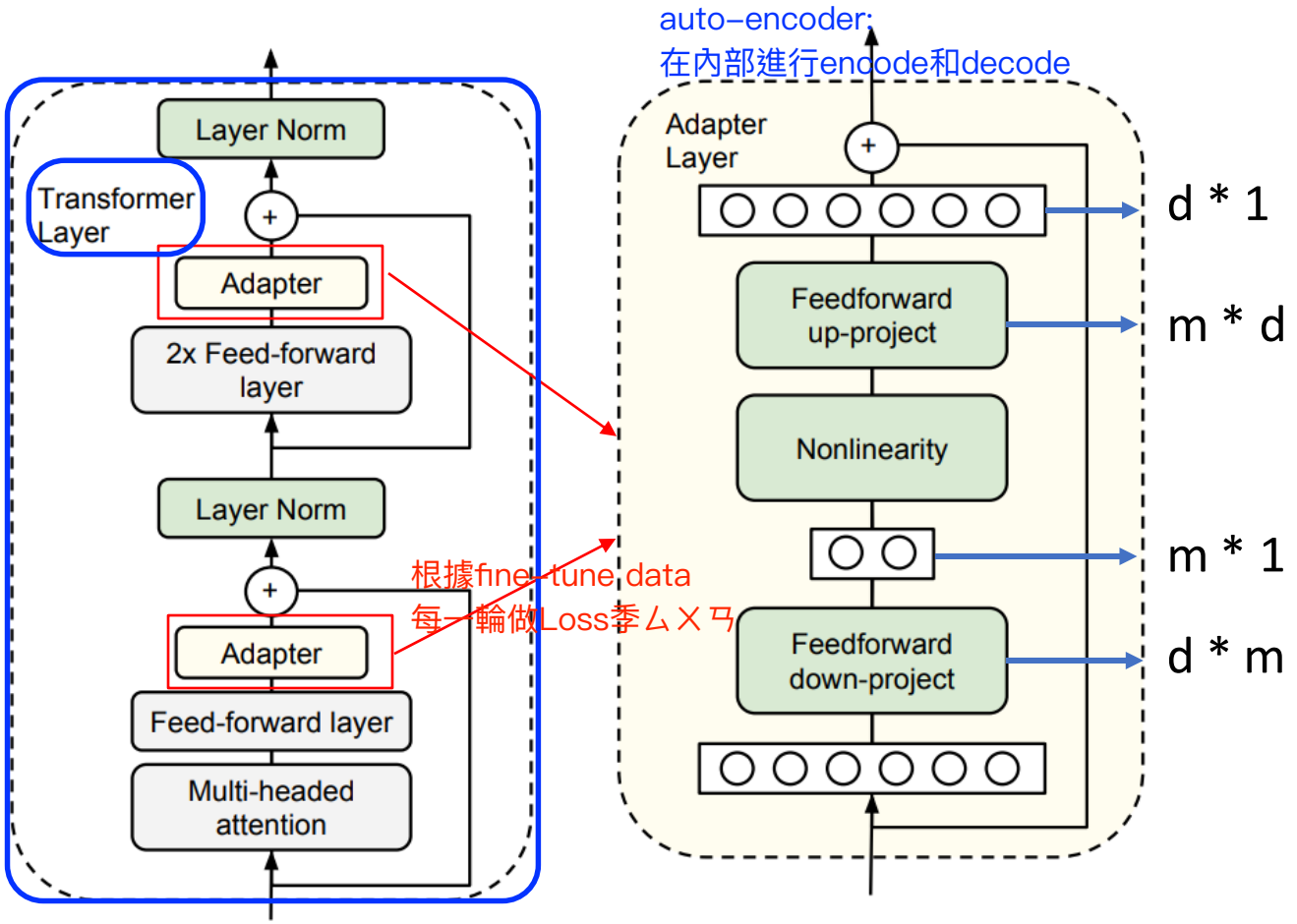
PEFT Current Development

- **Additive Method:**
  最後面新增額外參數，不改原模型
  在原本的pre-train model上，加上新的training layer 或module
  => 原模型權重通常保持frozen，只會train新增加的部分
  - Additive fine-tuning approaches involve introducing new extra trainable parameters for task-specific fine-tuning.
- **Selective Method:**
  只微調模型一部分的重要參數，從既有的參數中選出部分進行fine-tune
  =>只調整LayerNorm, bias或縱最後幾層layer的權重
  - Selective fine-tuning methods aim to reduce the number of fine-tuned parameters by selecting a subset of pre-trained parameters that are critical to downstream tasks while discarding unimportant ones.
- **Reparametrization-based Method:**
  透過數學分解壓縮參數結構，改變模型內部的參數表示方式
  =>減少可訓練參數dimension，但仍然保留原模型能力
  - Reparameterized fine-tuning methods utilize low-rank transformation to reduce the number of trainable parameters while allowing operating with high-dimensional matrices (e.g., pretrained weights).

PEFT Current Development



Additive PEFT: Adapters



- The adapters first project the original d-dimensional features into a smaller dimension, m, apply a nonlinearity, then project back to d dimensions.
- By setting m << d, we limit the number of parameters added per task

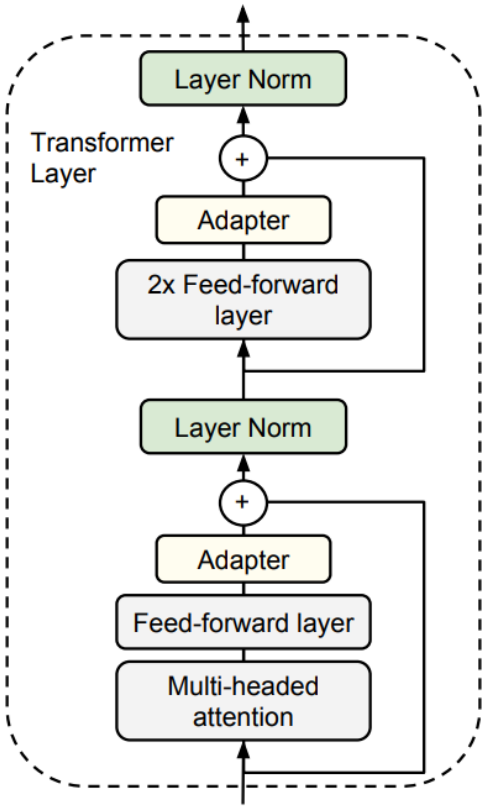
Houlsby, Neil, et al. "Parameter-efficient transfer learning for NLP." *International Conference on Machine Learning*. PMLR, 2019.

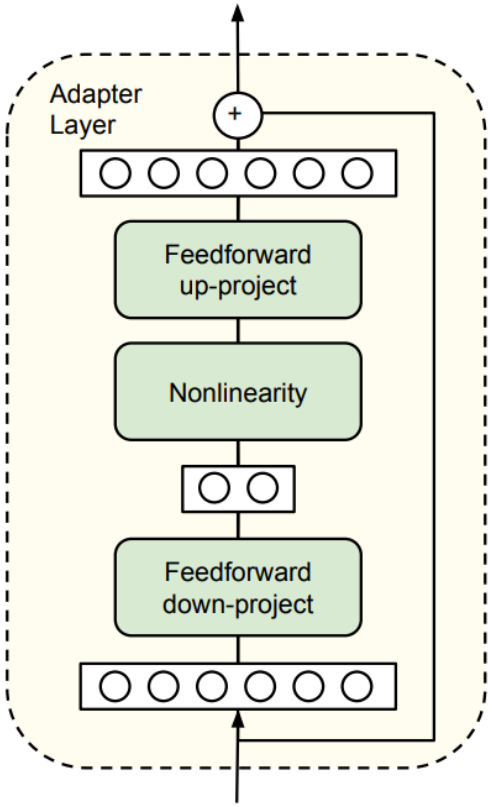
Additive PEFT: Adapters

Add fully-connected networks after attention and FFN layers

Pseudocode:

```
def transformer_block_with_adapter(x):
    residual = x
    x = SelfAttention(x)
    x = FFN(x)   # adapter
    x = LN(x + residual)
    residual = x
    x = FFN(x)   # transformer FFN
    x = FFN(x)   # adapter
    x = LN(x + residual)
    return x
```





Houlsby, Neil, et al. "Parameter-efficient transfer learning for NLP." *International Conference on Machine Learning*. PMLR, 2019.

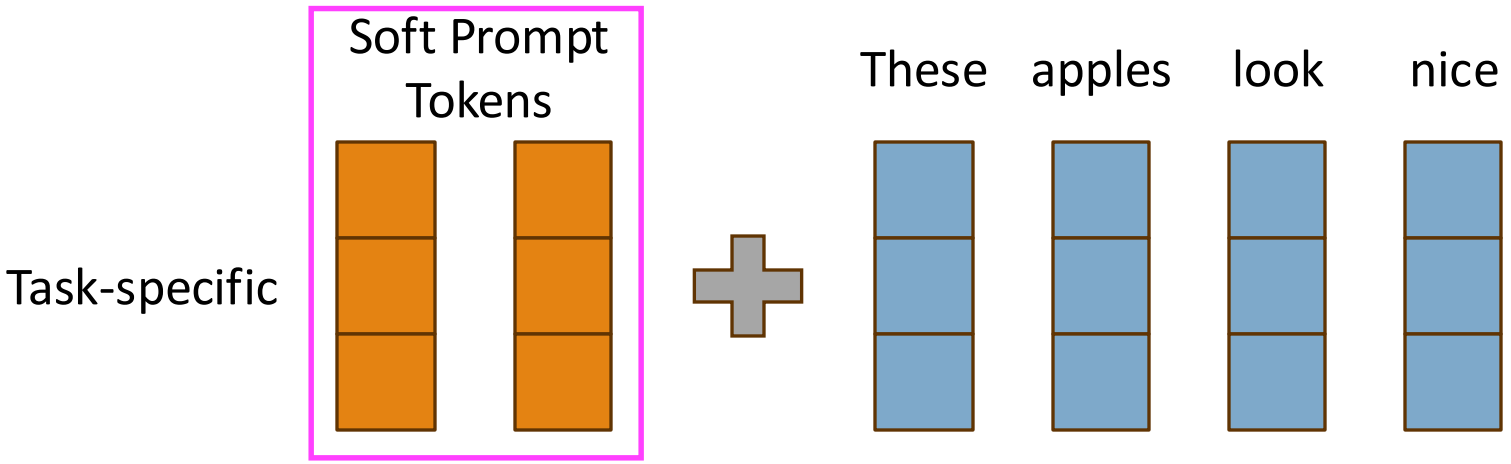
Additive PEFT: Prompt Tuning

寫給gpt的prompt在被進行優化一次

- Concatenates trainable parameters with the input embeddings
  - Learn a new sequence of task-specific embeddings
  - We call this prompt tuning because we only update prompt weights

人輸入的prompt: hard prompt
AI優化後的prompt: soft prompt

不影響模型本身



會先有pre-train資料先訓練好

Lester, Brian, Rami Al-Rfou, and Noah Constant. "The Power of Scale for Parameter-Efficient Prompt Tuning." *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021.

Additive PEFT: Prompt Tuning

- Efficient for multi-task serving
  - Each task is a prompt, not a model
  - Prompts for various tasks can be applied to different inputs

	Soft Prompt Tokens		Input embedding vectors			
Sentiment	[.6,…,-4.3]	[.2,…,5.4]	These	apples	look	nice
Q&A	[-1.2,…,.8]	[1.3,…,-2.7]	When	should	I	leave
Translate	[-.5,…,-1.3]	[.9,…,-.5]	I	love	fresh	air

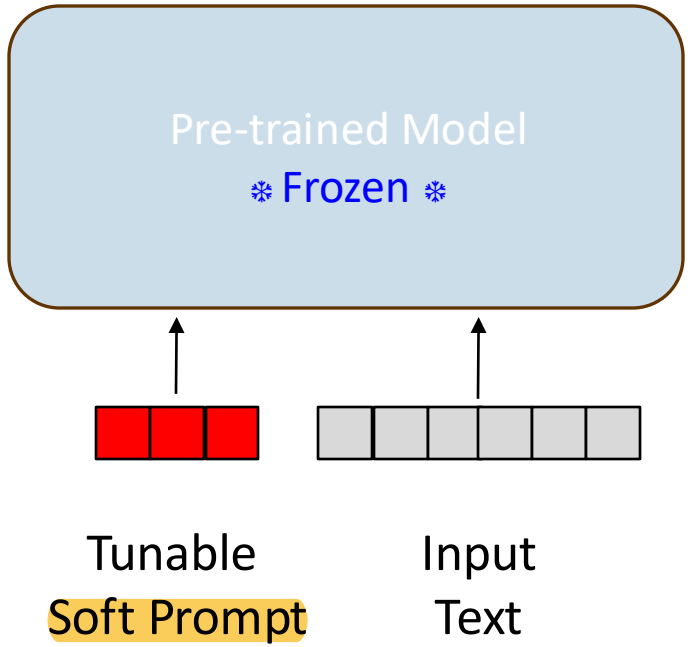
Lester, Brian, Rami Al-Rfou, and Noah Constant. "The Power of Scale for Parameter-Efficient Prompt Tuning." *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021.

Additive PEFT: Prompt Tuning

Prepend the model input embeddings with a trainable tensor

Pseudocode:

```
# make it a parameter so that it can be trained
# Initialize soft_prompt tensor with random values
soft_prompt = torch.nn.parameter(
    torch.rand(num_tokens, embedding_dim)
)
# Concatenate soft_prompt with input x
def input_soft_prompt(x, soft_prompt):
    x = concatenate([soft_prompt, x],
    dim = seq_len)

    return x

# train soft_prompt tensor with gradient descent
train(model(input_soft_prompt(x, soft_prompt)))

# use model with soft_prompt
model(input_soft_prompt(x, soft_prompt))
```



在input text進去的同時，soft prompt也會同時被加入進去

Lester, Brian, Rami Al-Rfou, and Noah Constant. "The Power of Scale for Parameter-Efficient Prompt Tuning." *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021.

https://arxiv.org/abs/2106.10199

Selective PEFT: BitFit

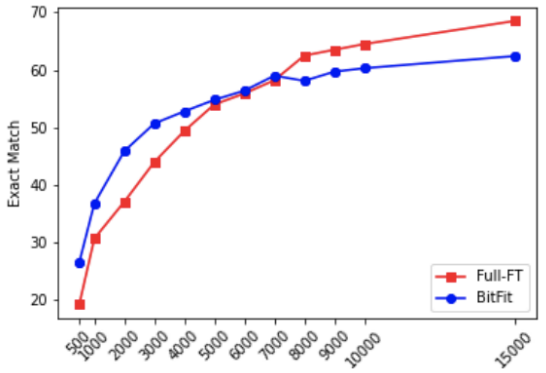
Fine-tune only model biases

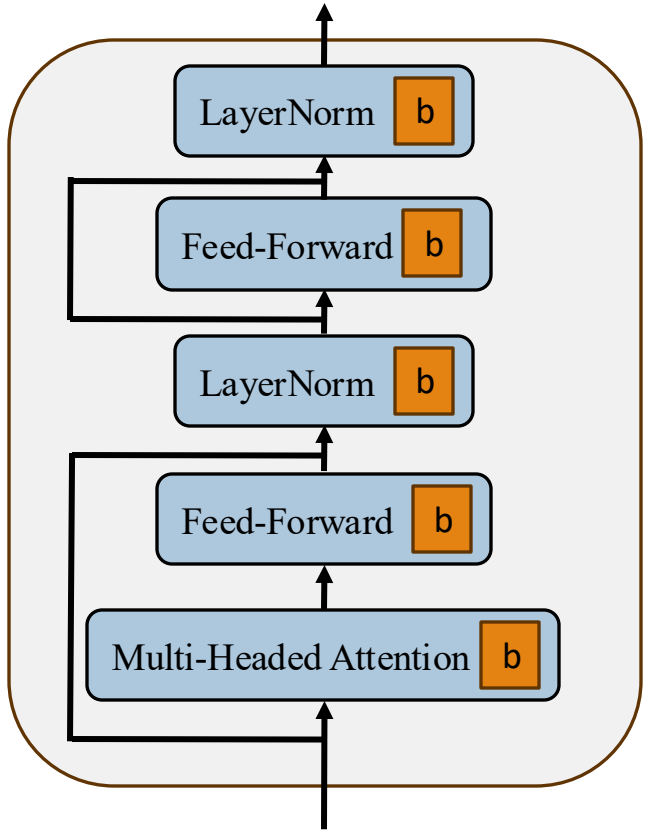
Pseudocode:

```
params = (p for n,p
          in model.named_parameters()
          if "bias" in n)
optimizer = Optimizer(params)
```

		%Param	QNLI	SST-2	$MNLI_m$	$MNLI_{mm}$	CoLA	MRPC	STS-B	RTE	QQP	Avg.
	Train size		105k	67k	393k	393k	8.5k	3.7k	7k	2.5k	364k	
(V)	Full-FT†	100%	**93.5**	**94.1**	**86.5**	**87.1**	**62.8**	**91.9**	89.8	71.8	**87.6**	**84.8**
(V)	Full-FT	100%	91.7±0.1	93.4±0.2	85.5±0.4	85.7±0.4	62.2±1.2	90.7±0.3	**90.0±0.4**	**71.9±1.3**	87.5±0.4	84.1
(V)	Diff-Prune†	0.5%	**93.4**	**94.2**	**86.4**	**86.9**	63.5	91.3	89.5	71.5	**86.6**	**84.6**
(V)	BitFit	0.08%	91.4±2.4	93.2±0.4	84.4±0.2	84.8±0.1	**63.6±0.7**	**91.7±0.5**	**90.3±0.1**	**73.2±3.7**	85.4±0.1	84.2
(T)	Full-FT‡	100%	91.1	**94.9**	86.7	85.9	**60.5**	**89.3**	**87.6**	70.1	**72.1**	**81.8**
(T)	Full-FT†	100%	**93.4**	94.1	86.7	**86.0**	59.6	88.9	86.6	**71.2**	71.7	81.5
(T)	Adapters‡	3.6%	90.7	94.0	84.9	85.1	59.5	89.5	**86.9**	71.5	**71.8**	81.1
(T)	Diff-Prune†	0.5%	**93.3**	94.1	**86.4**	**86.0**	**61.1**	**89.7**	86.0	70.6	71.1	**81.5**
(T)	BitFit	0.08%	92.0	**94.2**	84.5	84.8	59.7	88.9	85.5	**72.0**	70.5	80.9

Table 1: $BERT_{LARGE}$ model performance on the GLUE benchmark validation set (V) and test set (T). Lines with † and ‡ indicate results taken from Guo et al. (2020) and Houlsby et al. (2019) (respectively).





Zaken, Elad Ben, Yoav Goldberg, and Shauli Ravfogel. "BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models." *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 2022.

Reparametrization-based PEFT: LoRA

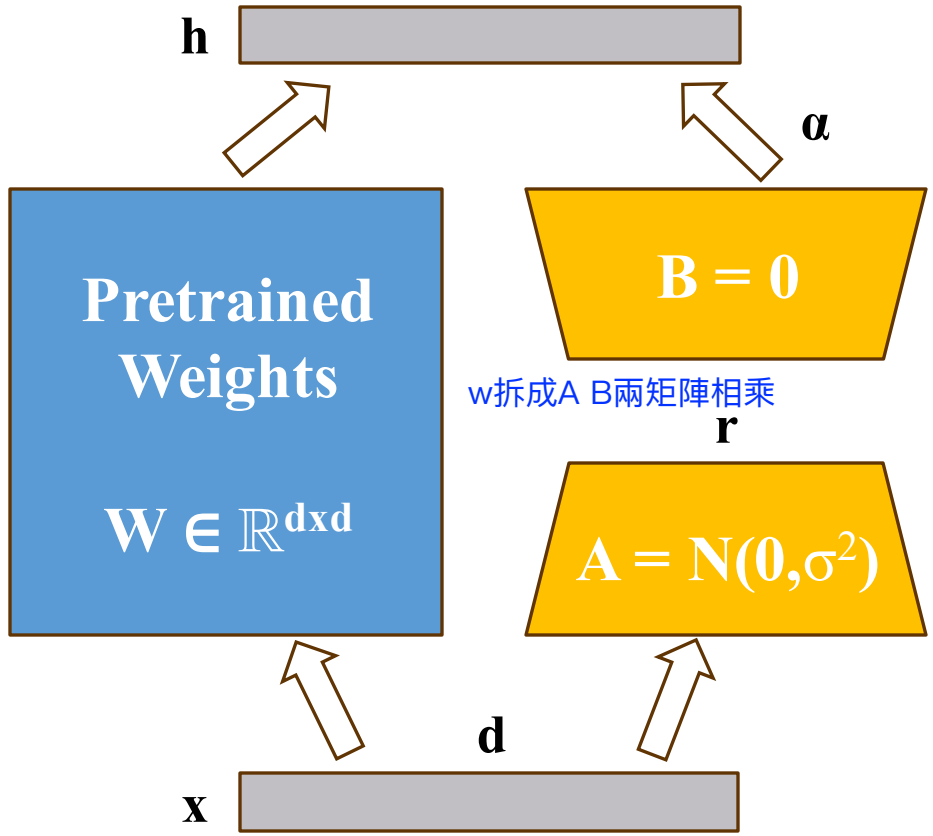
只針對更動的部分，用兩個矩陣相乘取代

$$h = W_0 x + \alpha \Delta W x$$
$$= W_0 x + \alpha B A x$$

$A \in \mathbb{R}^{r \times k}$ $B \in \mathbb{R}^{d \times r}$

Matrix rank　Hidden dimension　Input dimension

$\alpha \in \mathbb{R}^{+}$ ← Constant



Hu, Edward J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." *International Conference on Learning Representations*. 2021.

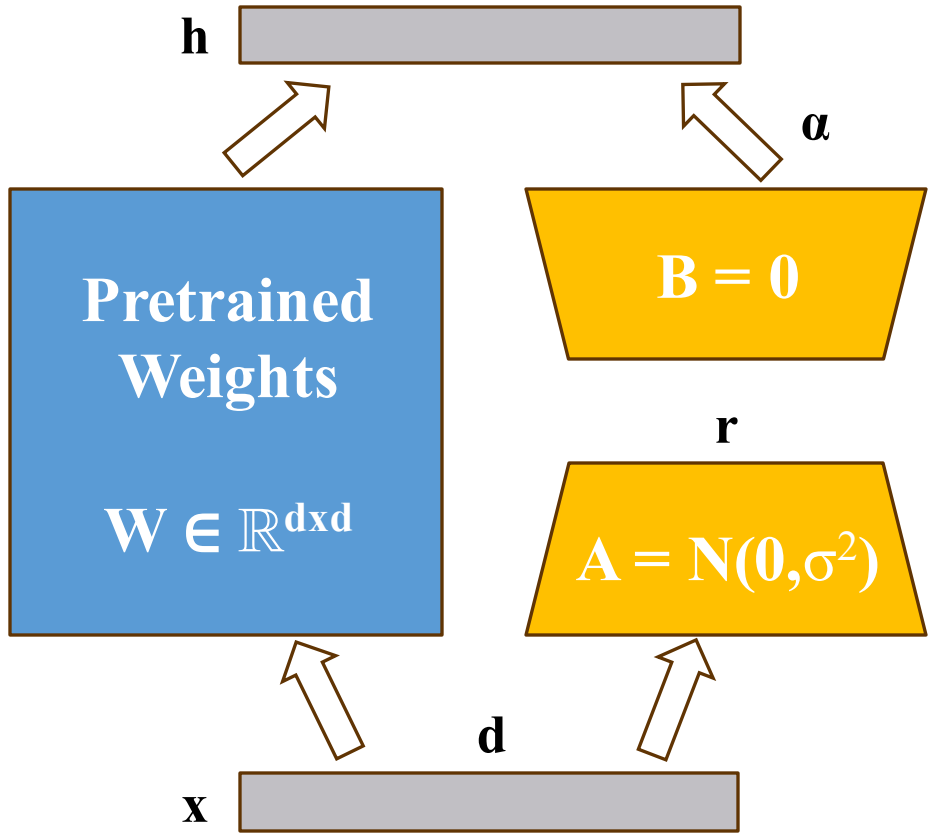
Reparametrization-based PEFT: LoRA

Decompose a weight matrix into lower-rank matrices

維度強制縮小

Pseudocode:

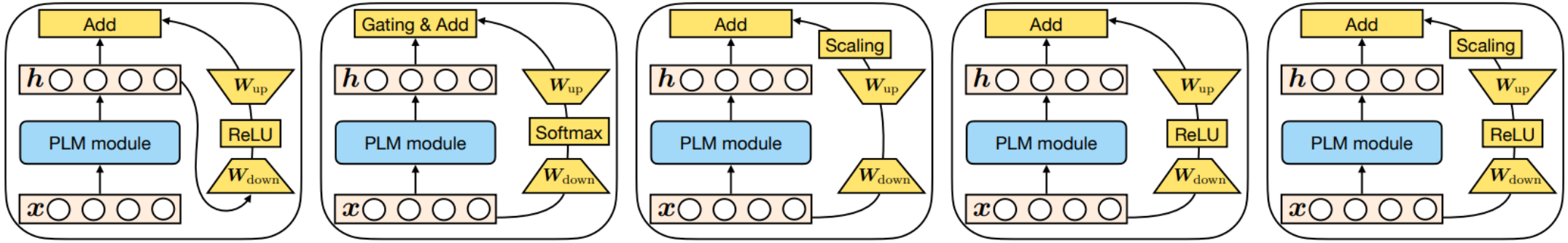
```
input_dim = 768  # the hidden size of the pre-trained model
output_dim = 768  # the output size of the layer
rank = 8  # The rank 'r' for the low-rank adaptation

W = ... # from pretrained network with shape input_dim x
output_dim

W_A = nn.Parameter(torch.empty(input_dim, rank)) # LoRA weight A
W_B = nn.Parameter(torch.empty(rank, output_dim)) # LoRA weight B

# Initialization of LoRA weights
nn.init.kaiming_uniform_(W_A, a=math.sqrt(5))
nn.init.zeros_(W_B)

def regular_forward_matmul(x, W):
    h = x @ W
return h

def lora_forward_matmul(x, W, W_A, W_B):
    h = x @ W  # regular matrix multiplication
    h += x @ (W_A @ W_B) * alpha # use scaled LoRA weights
return h
```



Hu, Edward J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." *International Conference on Learning Representations*. 2021.

Hybrid PEFT: MAM Adapters

- Break down the design of state-of-the-art parameter-efficient transfer learning methods and present a unified framework that establishes connections between them.



(a) Adapter (b) Prefix Tuning (c) LoRA (d) Parallel Adapter (e) Scaled PA

Ex. prompt tuning

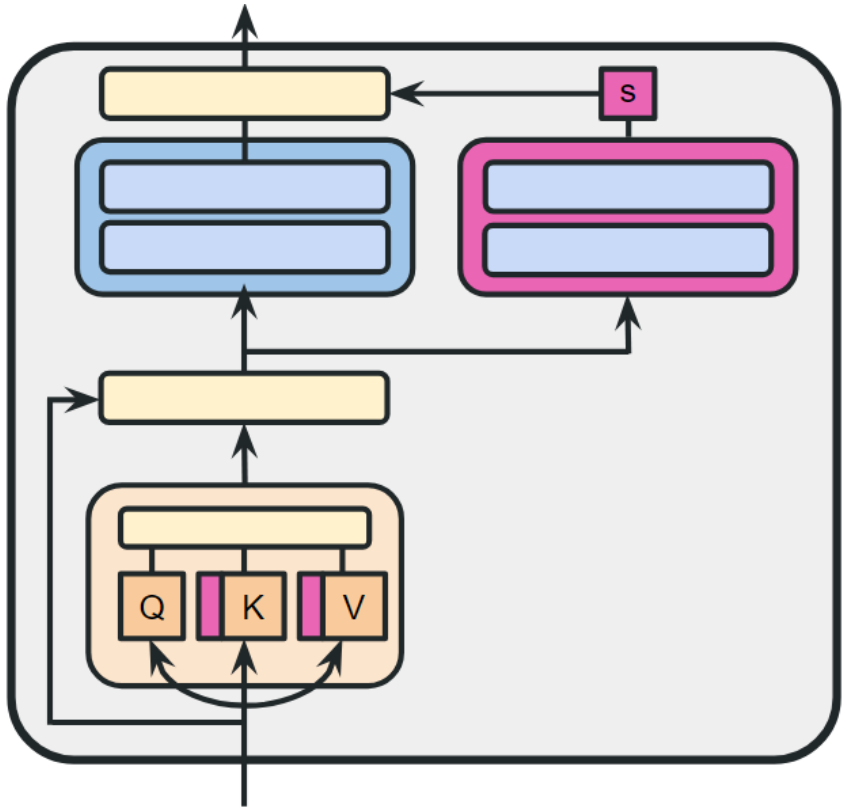
He, Junxian, et al. "Towards a Unified View of Parameter-Efficient Transfer Learning." *International Conference on Learning Representations*. 2021.

Hybrid PEFT: MAM Adapters

Scaled parallel adapter for FFN layer + soft prompt

Pseudocode:

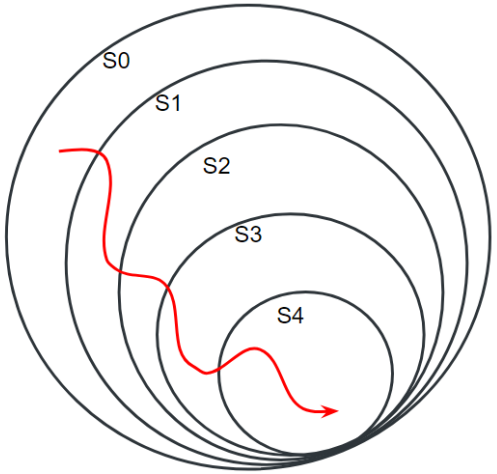
```
def transformer_block_mam(x):
    x =  concat([x,soft_prompt],
                dim=seq)
    residual = x
    x = SelfAttention(x)
    x = LN(x + residual)
    x_a = FFN(x) # parallel adapter
    x_a = scale * x_a
    x = LN(x + x_adapter)
    return x
```

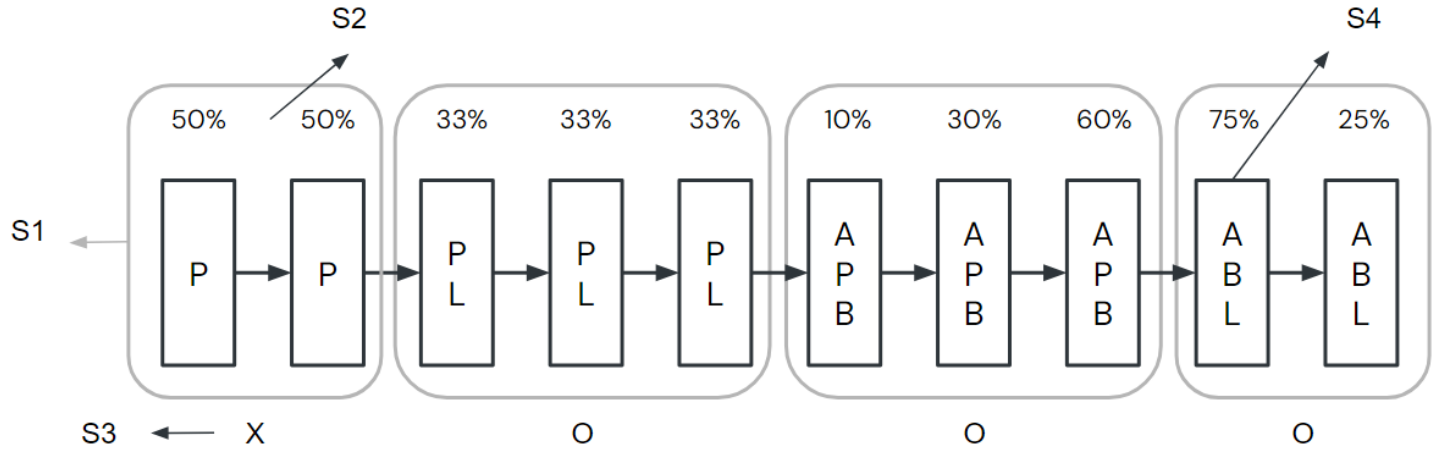


He, Junxian, et al. "Towards a Unified View of Parameter-Efficient Transfer Learning." *International Conference on Learning Representations*. 2021.

Hybrid PEFT: S4

- Designing Network Design Spaces
  - S0_The Initial Design Space: a random strategy
  - S1_Layer Grouping: Increasing, Uniform, Decreasing, Spindle, Bottleneck
  - S2_Trainable Parameter Allocation: Increasing, Uniform, Decreasing
  - S3_Tunable Groups: Tune or not
  - S4_Strategy Assignment: {Adapter (A), Prefix (P), BitFit (B), and LoRA (L)}





Chen, Jiaao, et al. "Parameter-Efficient Fine-Tuning Design Spaces." *The Eleventh International Conference on Learning Representations*. 2022.

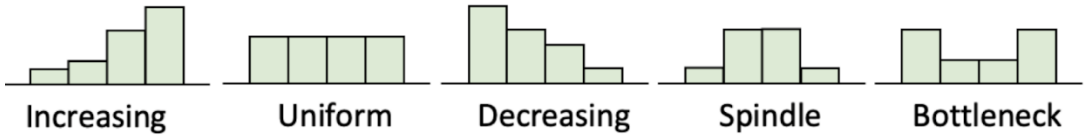
Hybrid PEFT: S4

Automatically found combination of Adapters (A), Prefix-Tuning (P), BitFit (B), and LoRA (L)

Search process:

At 0.1% additional parameters

1. Find optimal grouping pattern: spindle
2. Find optimal parameter allocation pattern: uniform
3. Find which groups need tuning: all
4. find optimal combinations of PEFT techniques sequentially for G1, G2, G3, G4



$$G_1 : A, L \quad G_3 : A, P, B$$
$$G_2 : A, P \quad G_4 : P, B, L$$

Chen, Jiaao, et al. "Parameter-Efficient Fine-Tuning Design Spaces." *The Eleventh International Conference on Learning Representations*. 2022.

PEFT Method: Summary

Method	Type	Storge	Memory	Infernce overhead	Trainable parameters	Changed parameters
Adapters	A	yes	yes	Extra FFN	0.1% - 6%	0.1% - 6%
Prompt Tuning	A	yes	yes	Extra input	0.1%	0.1%
Bitfit	S	yes	yes	Extra input	0.5%	0.5%
LoRA	R	yes	yes	No overhead	0.01% - 0.5%	0.5% - 30%
MAM Adapters	A	yes	yes	Extra FFN & input	0.5%	0.5%
S4	ASR	yes	yes	Extra FFN & input	0.5%	> 0.5%

Lialin, Vladislav, Vijeta Deshpande, and Anna Rumshisky. "Scaling down to scale up: A guide to parameter-efficient fine-tuning." *arXiv preprint arXiv:2303.15647* (2023).

What Matters in PEFT ?

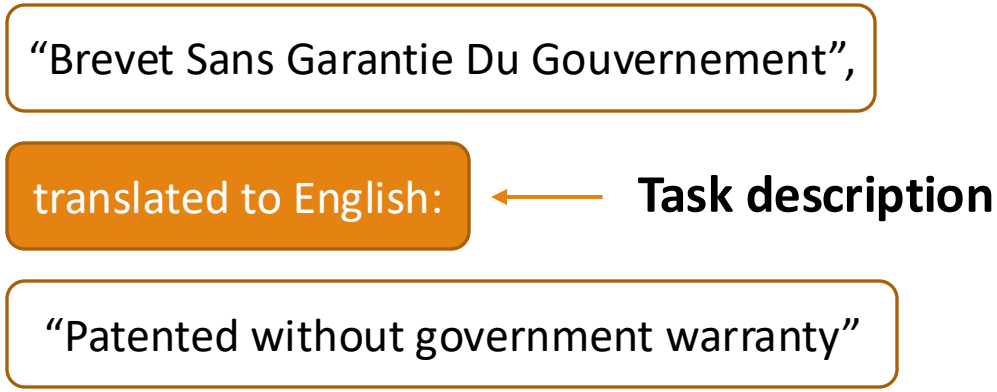
1. Number of parameters
2. Training efficiency
   - Do we add parameters to the network? How expensive are they?
   - Does the method require to backpropagate through the original network?
   - Can the method efficiently utilize the GPU?
3. Inference efficiency
   - Do we add parameters to the network? How expensive are they?
4. Accuracy
   - Do we get good performance out of the network in the end?

Prompt-based Learning

- Learning with **Hard Prompts** (**Discrete** Prompts)
- Learning with **Soft Prompts** (**Continuous** Prompts)

Downstream Tasks with Task Descriptions

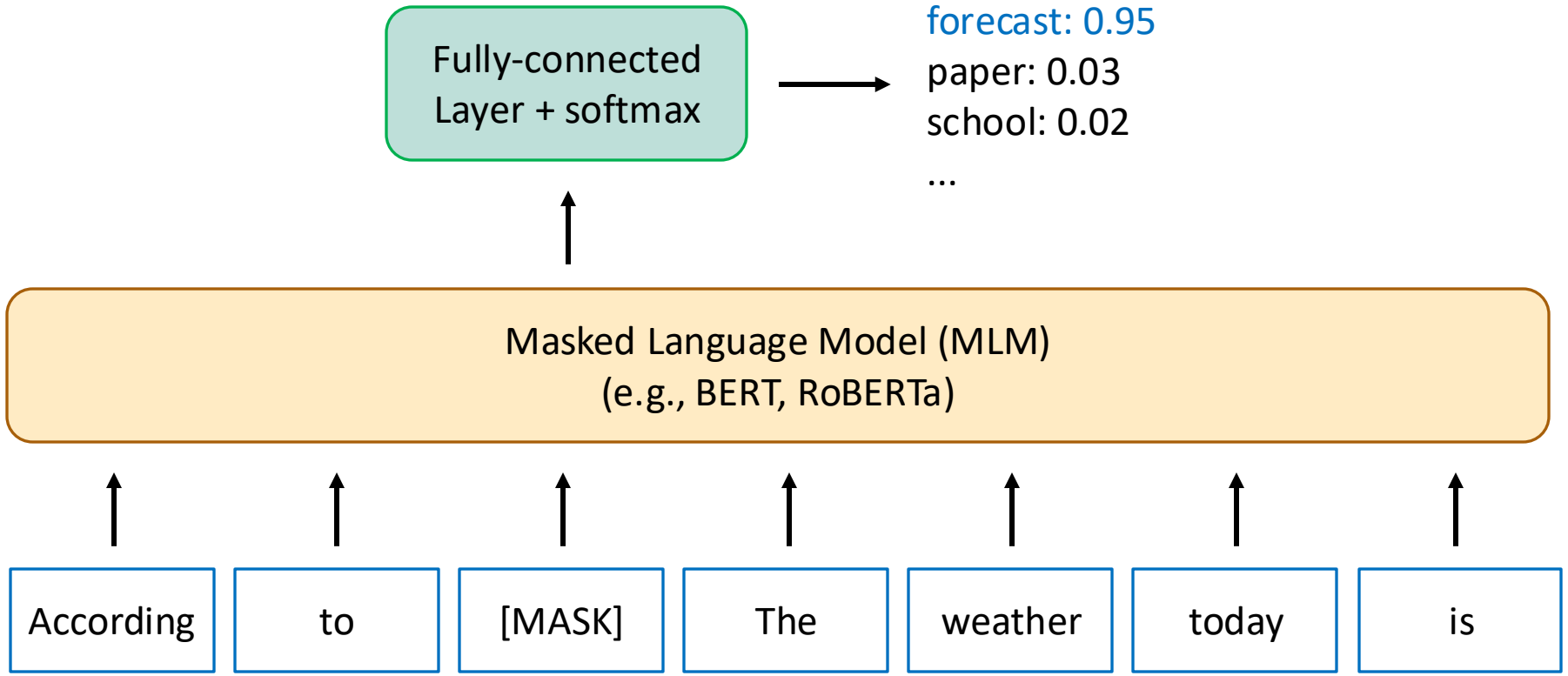
Take machine translation as an example



- Task descriptions = (discrete) prompts = demonstrations
- This idea was first proposed in the GPT-2 paper (Radford et al., 2019)[1].

[1] Radford, Alec, et al. "Language models are unsupervised multitask learners." *OpenAI blog* 1.8 (2019): 9.

Masked Language Modeling

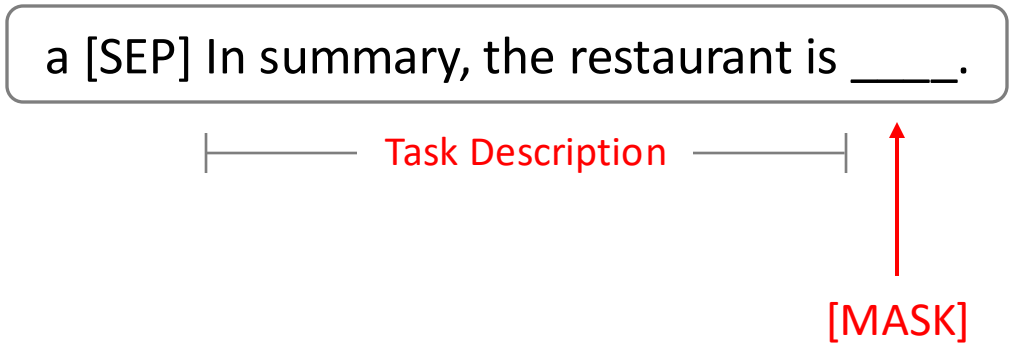


[2] Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.

Fine-tuning with Task Descriptions[3] (1/2)

Take Yelp Reviews (classifying a review into 1 to 5 points) as an example

Given a review a, the input can be transformed with a task description:



The transformed input can be used to fine-tune an MLM. However, typically, the answer of [MASK] should be text.

[3] Schick, Timo, and Hinrich Schütze. "Exploiting Cloze-Questions for Few-Shot Text Classification and Natural Language Inference." EACL 2021.

Fine-tuning with Task Descriptions[3] (2/2)

Take Yelp Reviews (classifying a review into 1 to 5 points) as an example

Given a review a, the input can be transformed with a task description and a verbalizer:

成效差–單純只有給積分
成效好–不給績分，而是給文字敘述

a [SEP] In summary, the restaurant is ____.

Task Description

[MASK]

Original Label in Yelp Reviews	Label for MLM with a verbalizer
1	terrible
2	bad
3	okay
4	good
5	great

[3] Schick, Timo, and Hinrich Schütze. "Exploiting Cloze-Questions for Few-Shot Text Classification and Natural Language Inference." EACL 2021.

Definition of a Verbalizer[3] and more examples[4]

- A verbalizer is a **one-to-one mapping** that maps each label to a word from M 's vocabulary. (Assume M is an MLM here.)
- Original labels are usually abstractive, while Verbalizers convert these labels into natural words.

Original Label in SST-2 [4]	Label for MLM with a verbalizer
positive	great
negative	terrible

Original Label in MNLI [4]	Label for MLM with a verbalizer
entailment	Yes
neutral	Maybe 通常會被移除
contradiction	No

[3] Schick, Timo, and Hinrich Schütze. "Exploiting Cloze-Questions for Few-Shot Text Classification and Natural Language Inference." EACL 2021.
[4] Gao, Tianyu, Adam Fisch, and Danqi Chen. "Making Pre-trained Language Models Better Few-shot Learners." ACL-IJCNLP 2021.

What's the advantage of fine-tuning with task descriptions?

- Fine-tuning with task descriptions forms classification tasks into generation tasks, which enables continuous training of a pre-training MLM directly without introducing classification layers. -> **fewer model parameters**
- Fine-tuning with task descriptions requires fewer training examples to reach similar performance of standard fine-tuning[3] .

[3] Schick, Timo, and Hinrich Schütze. "Exploiting Cloze-Questions for Few-Shot Text Classification and Natural Language Inference." EACL 2021.

Task descriptions also improves GPT-3 with in-context learning

- The three settings explored for in-context learning (ICL) in the GPT-3 paper[5]:

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← example
3  cheese =>                           ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← examples
3  peppermint => menthe poivrée        ←
4  plush girafe => girafe peluche      ←
5  cheese =>                           ← prompt
```

- Add task descriptions without fine-tuning -> ICL
- Fine-tuning with task descriptions -> Prompt-based fine-tuning

[5] Brown, Tom, et al. "Language models are few-shot learners." Advances in neural information processing systems 33 (2020): 1877-1901.

Problems of Tasks Descriptions

- There are vast combinations for discrete task descriptions. It is hard to search for the optimal one for a task[6].
- Searching the optimal task description for each task is computationally expensive[6].
- Discrete task descriptions cannot be optimized directly during training[7].

[6] Li, Xiang Lisa, and Percy Liang. "Prefix-Tuning: Optimizing Continuous Prompts for Generation." ACL 2021.
[7] Liu, Xiao, et al. "P-Tuning: Prompt Tuning Can Be Comparable to Fine-tuning Across Scales and Tasks." ACL 2022.

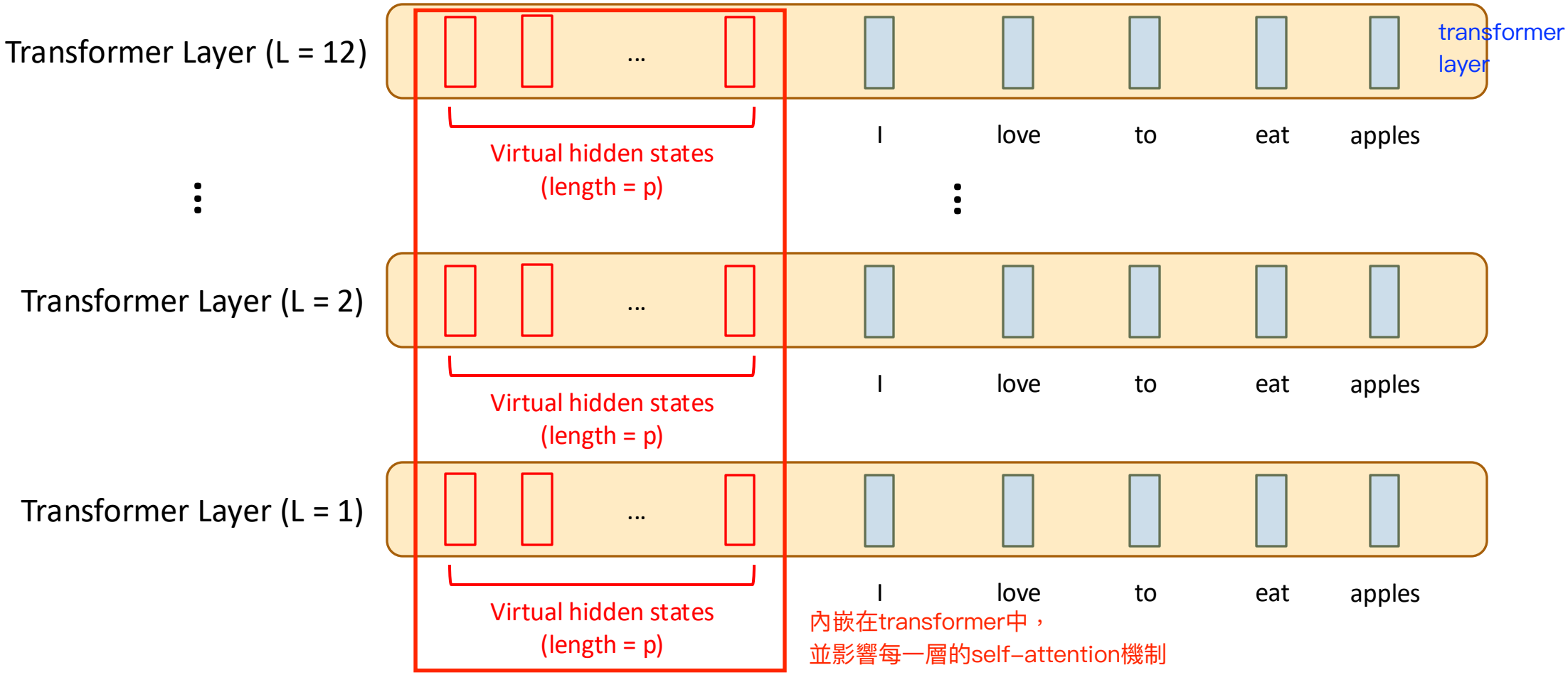
Prefix Tuning[6]

1.prompt tuning:
在embedding層加入可訓練的提示向量，不影響transformer
=> before: [CLS] I love this movie. /after: [Prompt1][Prompt2][Prompt3][CLS] I love this movie.
(其中PromptX為可學習向量，可以根據fine-tuning的任務進行調整)
2.Prefix tuning:
在Transformer的每一層attention模組/self-attention機制加入可學習的prefix key/value向量
=>[Prefix_K][Prefix_V] + [Original_K][Original_V]

- Prefix Tuning was proposed by Li and Liang, 2021[6] for generation tasks.
- p virtual hidden states (activations), which mimic virtual output of self-attention, are prepended at the left of the hidden states from the inputs for each layer.
- The pre-trained model (GPT-2 / BART) is frozen during training, while the p virtual hidden states in each layer are trainable.
- P-Tuning is same as Prefix Tuning, but P-Tuning was tested for natural language understanding tasks.

[6] Li, Xiang Lisa, and Percy Liang. "Prefix-Tuning: Optimizing Continuous Prompts for Generation." ACL 2021.
[7] Liu, Xiao, et al. "P-Tuning: Prompt Tuning Can Be Comparable to Fine-tuning Across Scales and Tasks." ACL 2022.

Illustration of Prefix Tuning



[6] Li, Xiang Lisa, and Percy Liang. "Prefix-Tuning: Optimizing Continuous Prompts for Generation." ACL 2021.
[7] Liu, Xiao, et al. "P-Tuning: Prompt Tuning Can Be Comparable to Fine-tuning Across Scales and Tasks." ACL 2022.

Implementation of Prefix Tuning[6]

- The initialized parameters of p virtual hidden states will be first reparametrized by a multi-layer perceptron network (MLP).
  - This technique stabilizes training of Prefix Tuning.
- The total number of trainable parameters include the p virtual hidden states from all layers and the parameters of MLP.

[6] Li, Xiang Lisa, and Percy Liang. "Prefix-Tuning: Optimizing Continuous Prompts for Generation." ACL 2021.

Parameter Initialization in Prefix Tuning

- In low-data settings:
  - Random initialization leads to low performance with high variance.
  - Initializing with task relevant words such as “summarization” and “table-to-text” obtains better performance.
- In full data settings: no difference.

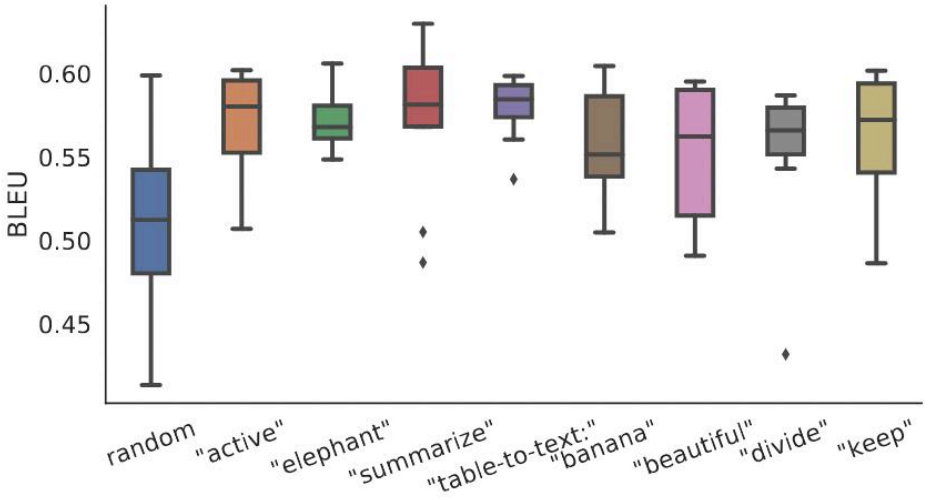
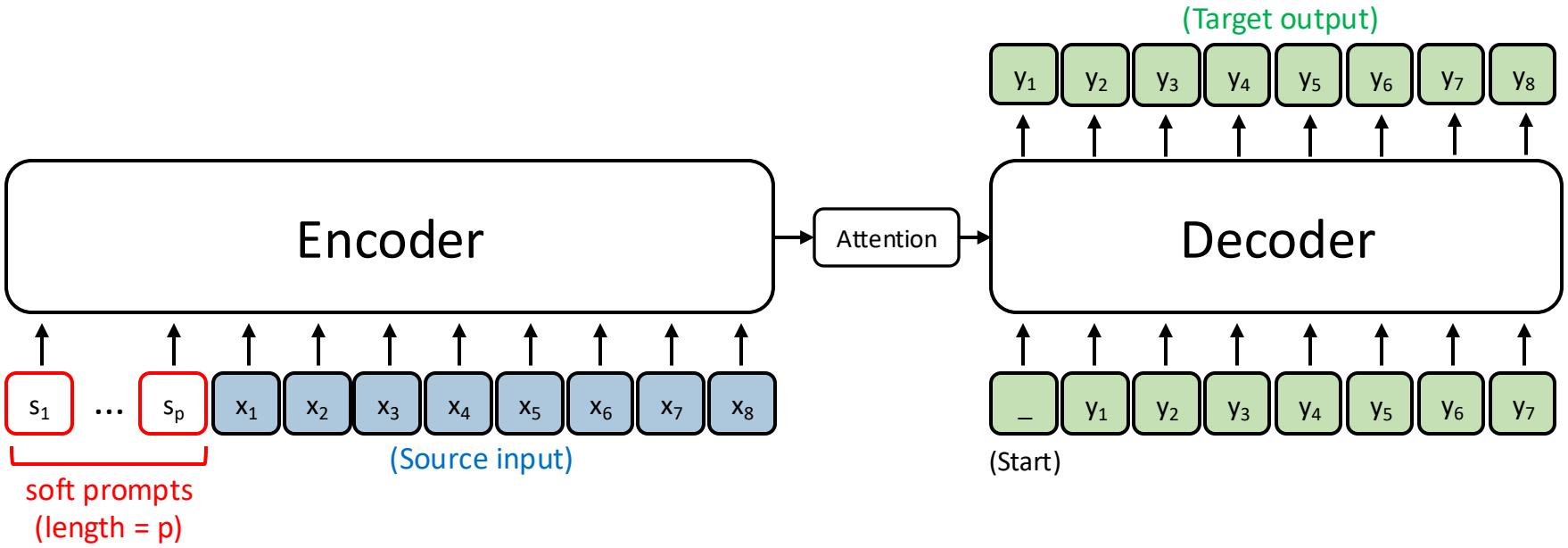


Figure 5: Initializing the prefix with activations of real words significantly outperforms random initialization, in low-data settings.

[6] Li, Xiang Lisa, and Percy Liang. "Prefix-Tuning: Optimizing Continuous Prompts for Generation." ACL 2021.

Soft Prompt Tuning

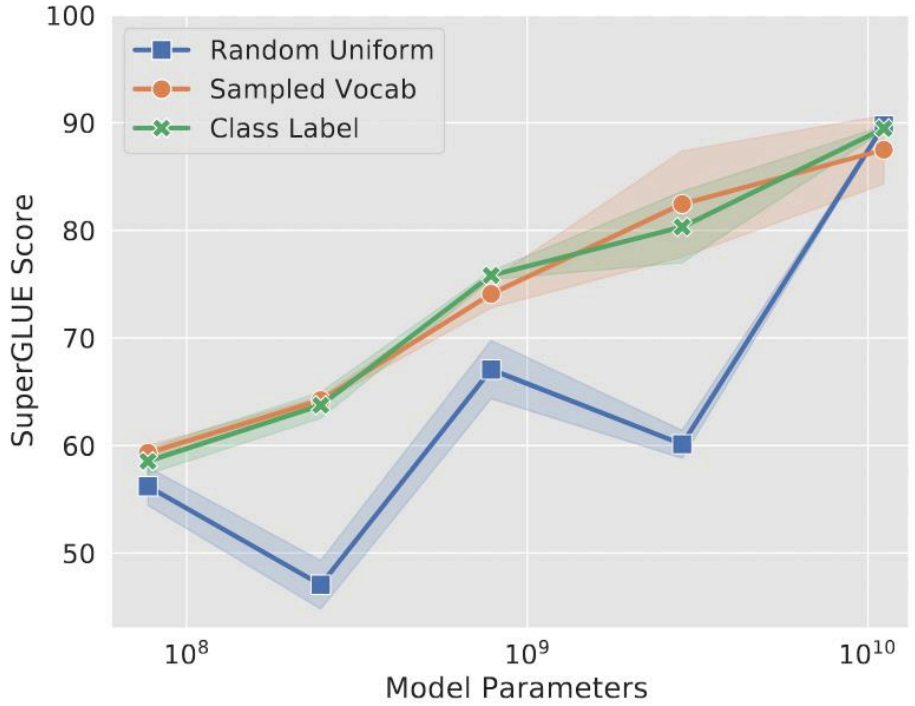
- Soft prompt tuning was proposed by Lester et al., 2021[8] .
- p tokens with trainable vectors are prepended to inputs for the pre-trained T5 model.
- The pre-trained T5 model is frozen during training.



[8] Lester, Brian, Rami Al-Rfou, and Noah Constant. "The Power of Scale for Parameter-Efficient Prompt Tuning." EMNLP 2021.

Parameter Initialization in Soft Prompt Tuning

- The class-based initialization performs best. (If p > num_labels, then sample from vocab.)
- At smaller model sizes, there are large gaps between the different initializations.
- Once the model is scaled to XXL size, those differences disappear.



(b) Prompt initialization

[8] Lester, Brian, Rami Al-Rfou, and Noah Constant. "The Power of Scale for Parameter-Efficient Prompt Tuning." EMNLP 2021.

Comparison between Prefix Tuning and Soft Prompt Tuning

	Prefix Tuning	Soft Prompt Tuning
Number of Trainable Parameters	Greater (Prefix_length * Hidden_size * number_layers)	fewer Prompt_length * Hidden_size
Inference Speed	Slower	Faster
Performance	Better	Worse
Use Cases	Better **Few-shot Learning** over Full Fine-tuning (With abundant training data, there is no difference in performance.)	